Module	Problem	Candidates	Selection	Justification
A. Relief inventory	Input: n crates (weight w_i 1 to 20, value v_i), capacity W. Output: subset with total weight <= W and maximum total value (0/1 knapsack).	Dynamic programming over capacity, O(nW). Meet in the middle, O(n 2^(n/2)). Exhaustive search, O(2^n). Greedy by value per kg, O(n log n), not exact.	Primary: DP table O(nW) with traceback. Baseline: exhaustive search on tier S (also checks the claims); greedy for comparison.	Weights are integers up to 20 and W is at most 0.7 of total weight, so nW stays near 31 million on tier L. Fails if weights were huge or real valued (W too large); then use meet in the middle. Case 47 shows greedy reaching under 60% of optimum. Case 46 is already sorted, so the greedy sort must not use a first element pivot.
B. Route planning	Input: network V locations, E roads, source HQ, today's blocked roads. Output: cheapest route and cost (step cost = minutes + penalty of arrival), or none; reachable set; fewest-roads route.	Dijkstra with binary heap, O((V+E) log V). Bellman-Ford, O(VE). BFS for reachability and fewest roads, O(V+E). Dijkstra with array, O(V^2).	Primary: Dijkstra (heap) for cost plus BFS for hops and reachability. Baseline: Bellman-Ford.	Costs are at least 1 (minutes >= 1), so no negative edges, and E <= 3V so the heap beats the O(V^2) array. Fewest roads and cheapest differ (case 48, 1.5x or more), so two searches are needed. Fails with negative costs; closures can disconnect the target, so return none. Chains reach 500+ roads deep: use iterative search, not recursion.
C. Rescue scheduling	Input: n windows (start, finish, value). Output: pairwise compatible set (touching allowed), maximum count or maximum total value.	Earliest finish greedy, O(n log n). Weighted interval DP with binary search, O(n log n). Exhaustive search, O(2^n).	Primary: greedy by earliest finish for count, sorted DP for value. Baseline: exhaustive search (tier S).	Windows lie in 0 to 1440, so sorting by finish is cheap, and the data has touching pairs, so compatibility must use finish <= start. Greedy is exact for count only; it fails for value. In study 3 windows overlap heavily, so the DP cannot be skipped.
D. Emergency queue	Input: n alarms (tick, priority 1 to 10). Output: release order, one alarm per tick, most urgent among those present (ties by earlier alarm_id).	Binary heap, O(n log n). Linear scan of pending list, O(n q). Bucket queue by priority (10 buckets), O(n).	Primary: binary heap keyed (priority, alarm_id). Baseline: linear scan.	On tiers M and L arrivals are 1.2 to 1.8 per tick against one release per tick, so the backlog grows and scanning it costs O(n^2). Bursts (study 2) make the backlog spike. Fails if the key ignores ties, or if the queue is not drained after the last tick.
E. Radio logs	Input: log of N symbols (38 letters), phrase of length m, two reports of lengths a and b. Output: all start positions (overlaps included); minimum insertions, deletions, replacements with an edit script; prefix-free binary code of minimum total length.	Search: KMP O(N+m); Rabin-Karp O(N+m) expected; naive O(Nm). Edit: DP O(ab) time and space; Hirschberg O(ab) time, O(min(a,b)) space. Code: Huffman with heap, O(N + s log s), s <= 38.	Primary: KMP, edit DP with traceback, Huffman. Baseline: naive search, Hirschberg.	Case 49 is 90% repeats of one block, so near-miss phrases make naive search cost about Nm. Reports are at most 1000 symbols, so the 1M cell DP fits in memory. One symbol is 25% of the log in study 6 (skewed code). Fails: Rabin-Karp on hash collisions; Huffman with a one-symbol log needs a 1 bit code.
F. Network planning	Input: roads with cable cost, reachable locations after today's closures. Output: set of roads linking all reachable locations with minimum total cable cost.	Kruskal with union-find, O(E log E). Prim with heap, O(E log V). Boruvka, O(E log V).	Primary: Kruskal. Baseline: Prim.	The graphs are sparse (E <= 3V), where sorting E edges is cheap, and cable costs 1 to 100 give many ties, so the tree is not unique but the total is. Fails on dense graphs (Prim with array wins); closures can split the map, so build a tree only over the component of the HQ.
G. Chains and cuts	Input: dims p_0..p_n (n matrices), drum length L with prices. Output: bracketing with fewest scalar multiplications (cost p q r); best total price of cuts.	Chain: DP O(n^3), memoised recursion O(n^3), exhaustive over Catalan(n) bracketings. Drum: DP O(L^2), recursion O(2^L), greedy by price per length O(L log L).	Primary: bottom-up DP for both. Baseline: exhaustive (small n, L); greedy price per length for the drum.	n <= 300 gives about 4.5 million inner steps and L <= 2000 about 2 million. Prices are not proportional to length (data rule), so greedy by price per length fails. Fails only if n or L grew so that O(n^3) is too slow; use Hu-Shing then.
H. Hard planning questions	Input: 3-SAT rules over n switches; ration values and target T; sites; sensor graph and k. Output: satisfying assignment or none; subset summing to T or none; shortest closed tour; cover of at most k nodes or none.	SAT: DPLL with unit propagation, O(2^n) worst; WalkSAT; brute force O(2^n m). Subset sum: DP O(nT); meet in the middle O(2^(n/2)). Tour: Held-Karp O(n^2 2^n); 2-opt heuristic; brute force O(n!). Cover: bounded search tree O(2^k m); brute force O(C(n,k)).	Primary: DPLL; subset sum DP; Held-Karp (n <= 16) and nearest neighbour plus 2-opt (n <= 500); bounded search tree with degree rules. Baseline: brute force, WalkSAT.	Rules per switch run 2.5 to 6.0, with many cases at 3.5 to 4.8 near the hard threshold, so DPLL branches deeply but n <= 50. Planted solutions guarantee that answers exist, but unsatisfiable inputs must still be refuted. Fails: subset sum DP if T is huge; Held-Karp beyond n = 20; WalkSAT cannot prove none.
I. Nearest sites and checksums	Input: n sites (x, y in 0 to 999); two long decimal numbers (d digits) built from crate serials. Output: pair at minimum squared Euclidean distance; their product.	Closest pair: divide and conquer O(n log n); grid hashing O(n) expected; brute force O(n^2). Product: Karatsuba O(d^1.585); FFT O(d log d); schoolbook O(d^2).	Primary: divide and conquer; Karatsuba. Baseline: brute force; schoolbook.	Sites are distinct integers, so the minimum distance is at least 1 and the strip step is safe; n <= 500 is small enough that brute force is a fair baseline. Serials make numbers of up to 24,000 digits, where schoolbook is too slow. Fails: many collinear sites make strips large; Karatsuba loses on short numbers (use a cutoff).

Member	Contribution
Ali Asjad 23I-2648, Section A	Generator core: crates, locations, roads, closure waves, signature rules for studies 1, 4, 5; traps 46 and 48. Validator checks V1 to V4 and V8 for traps 46 and 48. Approach note part 1. Repository set up. Proposes for Iteration 2: A Relief inventory, B Route planning, F Network planning.
Sameer Khan 23I-2521, Section A	Generator: radio log, patterns, reports, windows, incidents, claims, studies 2, 3, 6, trap 49. Validator checks V5 and the V7 signature rules for studies 2, 3, 6; broken test copies. Approach note parts 3 and 4. Proposes for Iteration 2: C Rescue scheduling, D Emergency queue, E Radio logs.
Saad bin Saed 23I-6050, Section A	Generator: sites and planted tour, pipeline and drum, sensor graph and cover, safety rules and planted assignment, rations, trap 47. Validator checks V6, V7 variety rules, V8 for traps 47 and 49. Approach note part 2, validation report. Proposes for Iteration 2: G Chains and cuts, H Hard planning, I Nearest sites and checksums.